

Lesson 3 — Making It Move

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The loop that never waits

```
def game_loop():
    update()
    draw()
    screen.update()
    screen.ontimer(game_loop, 16)  # call
    me again

game_loop()
screen.mainloop()
```

16 ms $\approx$ 1/60 second $\approx$ 60 fps.

This is the web track's `requestAnimationFrame`.
Same idea, different word.

Keys: record, never move

```
keys = {}

def press(name):
    def handler():
        keys[name] = True
    return handler

def release(name):
    def handler():
        keys[name] = False
    return handler

for name in ["Up", "Down", "Left", "Right"]:
    screen.onkeypress(press(name), name)
    screen.onkeyrelease(release(name), name)

screen.listen()  # OR NOTHING WORKS
```

Then in `update()`:

```
if keys.get("Right"):
    player_x += SPEED
```

`keys.get(name)` — `keys[name]` raises `KeyError` for an untouched key.

Why `press(name)` returns a function

A `lambda` inside the loop shares **one** `name`, so every key ends up setting the last one. That is the **closure trap**, and it produces a bug with no error message.

The three silent failures

Missing	Symptom
<code>screen.listen()</code>	no key does anything
<code>drawer.clear()</code>	everything smears
<code>onkeyrelease</code>	the player never stops

Clamping

```
x = max(-330, min(330, x))
```

Inside out: never more than 330, never less than -330.

Delta time (the proper way)

```
now = time.time()
dt = now - last_time
last_time = now
if dt > 0.1:
    dt = 1 / 60  # clamp
player_x += SPEED * dt  # SPEED is px per
SECOND
```

Without it, a faster computer plays a faster game.

Words

Polling — asking every frame. **Key repeat** — the OS re-sending a held key. **Closure** — a function remembering where it was made.

Section A — Recall

A1. What is the single most important difference between the lesson 1 loop and the lesson 3 loop? [3 marks]

A2. What does `screen.ontimer(game_loop, 16)` do, and which JavaScript function does the same job? [2 marks]

A3. Why does moving the player inside the key handler make it stutter? [2 marks]

A4. Name the three things that, if missing, cause a silent failure in this lesson — and give the symptom of each. [3 marks]

A5. Why `keys.get("Right")` rather than `keys["Right"]`? [2 marks]

Section B — Predict the output

B1. All four arrow keys make the player move in the *same* direction. Why, and what is this mistake called? [4 marks]

```
for name in ["Up", "Down", "Left", "Right"]:  
    screen.onkeypress(lambda: keys.update({name: True}), name)
```

B2. `SPEED` is 5 pixels per frame and `ontimer` asks for 16 ms. How many pixels per second is that? [3 marks]
What happens to the real speed if the computer only manages 30 frames per second?

B3. The player holds Up and Right together, with `SPEED = 5`. How far do they actually travel each frame? Show the working, and say by what percentage that is faster than moving straight. [4 marks]

B4. What does the player see?

[3 marks]

```
def game_loop():
    update()
    draw()
    screen.update()
    # no ontimer line
```

Section C — Find and fix the bug

C1. The window opens, the player is drawn, and no key does anything at all. There is no error. What is missing? [3 marks]

C2. The player leaves a solid snake-like trail of circles behind it. What is missing, and from which function? [3 marks]

C3. This crashes with `KeyError: 'Right'` the moment the game starts — before any key has been pressed. Explain why, and give two different fixes. [4 marks]

```
def update():
    global player_x
    if keys["Right"]:
        player_x += SPEED
```

Section D — Build it

- ☐ **Checkpoint 1.** Get a `game_loop()` running with `ontimer`. Prove it is running by drawing a number that counts up every frame.
- ☐ **Checkpoint 2.** Add the `keys` dictionary with `onkeypress` and `onkeyrelease`. Draw the list of held keys on screen. Do not move anything yet.
- ☐ **Checkpoint 3.** Draw a player circle and make the arrow keys move it — **inside** `update()`. Hold a key: it should glide with no pause.
- ☐ **Checkpoint 4.** Clamp the player inside the window. Test by holding a key for several seconds.
- ☐ **Checkpoint 5.** Add WASD as a second control scheme using the *intent* pattern, so `update()` never mentions a key name directly.

- ☐ **Checkpoint 6.** Add a second circle that moves a little towards the player every frame. You have just written an enemy.

THREE THINGS TO CHECK BEFORE ASKING FOR HELP

1. Is `screen.listen()` there? 2. Is `drawer.clear()` the first line of `draw()`? 3. Is `screen.mainloop()` at the very end? These three cause almost every "it does nothing" problem in this lesson, and none of them produces an error message.

Section E — Design challenge

NO RIGHT ANSWER

E1. Holding Up and Right makes the player about **41% faster** than holding one key. Describe **three different** ways to fix it, and say which you would choose and why.

E2. Your player stops dead the instant you release a key. Think about a spaceship, a car and a mouse cursor. Which should stop instantly, which should not, and what makes the difference?

E3. You asked for a frame every 16 ms. What happens if `update()` and `draw()` together take 20 ms? Does the game run at 60 fps, 50 fps, or something else? What would you *measure* to find out?

E4. Your loop now runs whether or not the player does anything. **List five things that could happen in it that have nothing to do with the player** — and notice that none of them were possible in lesson 1.

Stretch goals

1. Fix the diagonal-speed bug. Say which approach you chose.
2. Momentum: a speed that builds and decays rather than snapping on and off.
3. Convert to proper delta time using `code/04-delta-time.py`, and draw the measured fps on screen.
4. A chaser that follows you. Then make it *avoidable* — much harder than it sounds.

5. Wrap around the edges instead of clamping. Which suits which kind of game?

Game Dev Bootcamp — Lesson 3 — Making It Move. Licensed MIT: copy it, change it, share it.